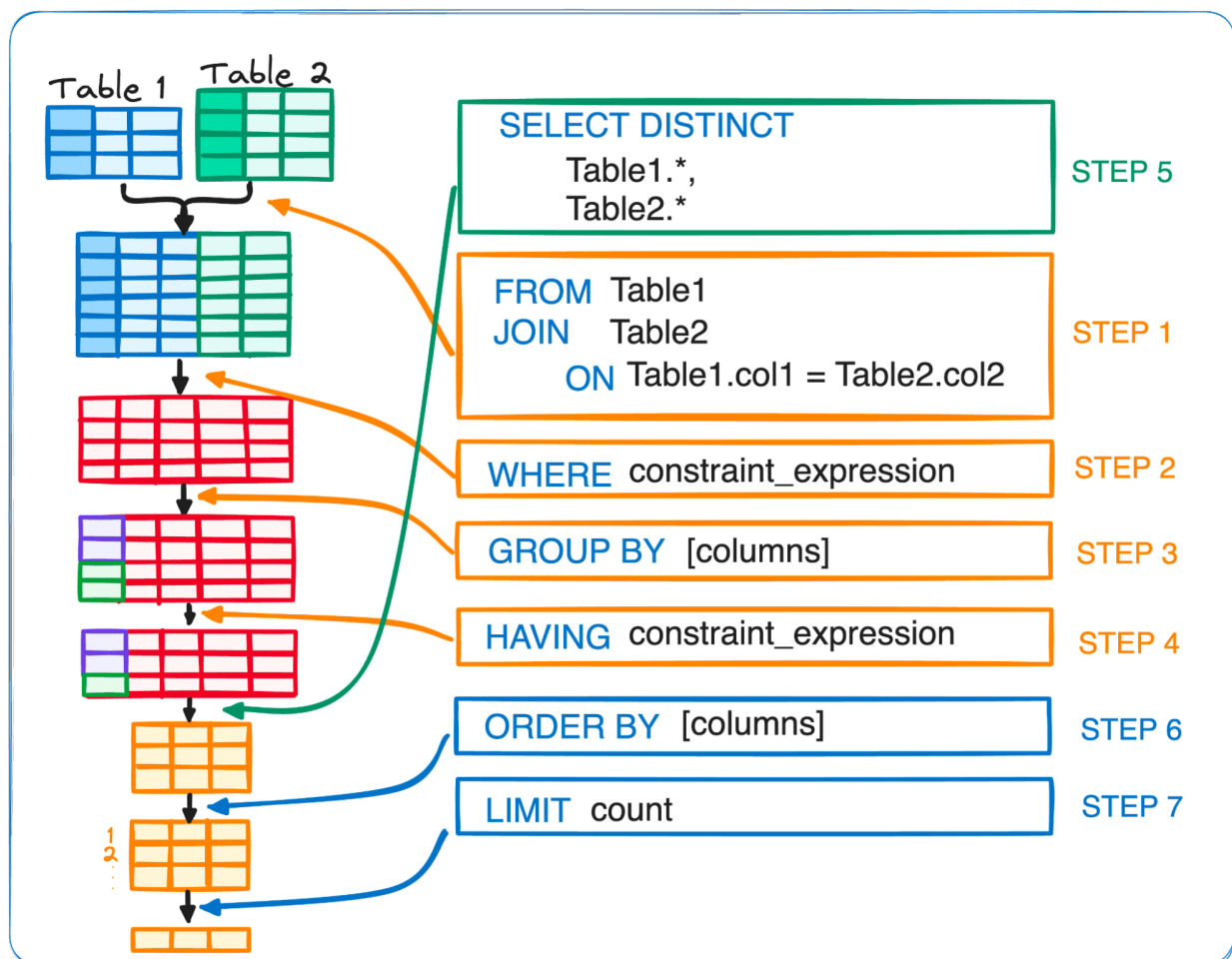


SQL Order of Execution

Lỗi thường gặp

1. Dùng **WHERE** với biểu thức tổng hợp (**aggregated**)
2. Dùng alias trong cùng SELECT trước khi nó được định nghĩa
3. Dùng **ORDER BY** với alias không tồn tại hoặc bị nhầm với biểu thức gốc
4. Gộp nhầm **GROUP BY** với cột không liên quan hoặc thừa/thiếu cột không nằm trong aggregate
5. Tin rằng **WHERE** và **HAVING** là thay thế cho nhau
6. Tin rằng thứ tự viết trong SQL tương đương thứ tự thực thi
7. Dùng **DISTINCT** và **GROUP BY** cùng lúc khi không cần thiết
8. Không hiểu sự khác biệt giữa **INNER JOIN** vs **LEFT JOIN** dẫn đến mất dữ liệu



Thứ tự	Clause	Chức năng chính	Ghi chú đặc biệt
1	FROM	Xác định bảng dữ liệu nguồn. Nếu có JOIN , sẽ thực hiện kết hợp bảng tại bước này.	Là bước đầu tiên, không được dùng alias từ SELECT .
2	JOIN	Thực hiện kết nối các bảng với nhau dựa trên điều kiện ON .	Có thể thực hiện INNER , LEFT , RIGHT , FULL JOIN.
3	WHERE	Lọc dòng dữ liệu trước khi thực hiện GROUP BY . Chỉ áp dụng cho các cột gốc (không phải cột alias, không phải giá trị tổng hợp).	Không được sử dụng alias hoặc hàm tổng hợp như AVG() , SUM() tại đây.
4	GROUP BY	Gom nhóm các dòng dữ liệu theo cột chỉ định. Bắt buộc khi có dùng các hàm tổng hợp mà vẫn muốn hiển thị cột khác.	Các cột không thuộc hàm tổng hợp phải nằm trong GROUP BY .
5	HAVING	Lọc dữ liệu sau khi nhóm (GROUP BY). Chuyên dùng để lọc điều kiện có tính toán tổng hợp (AVG() , COUNT() , ...).	Có thể dùng alias và hàm tổng hợp tại đây.
6	SELECT	Trả về các cột kết quả. Có thể dùng alias, biểu thức tính toán, hàm tổng hợp.	Alias chỉ được sử dụng sau SELECT , không được dùng ở WHERE .
7	DISTINCT	Loại bỏ các dòng trùng lặp sau khi SELECT .	Áp dụng sau SELECT , trước ORDER BY .
8	ORDER BY	Sắp xếp kết quả theo cột hoặc biểu thức. Có thể dùng alias đã định nghĩa ở SELECT .	Là bước duy nhất có thể sử dụng alias trong điều kiện sắp xếp.
9	LIMIT / OFFSET	Giới hạn số dòng kết quả (LIMIT) và số dòng bỏ qua (OFFSET).	Hỗ trợ bởi PostgreSQL, MySQL. SQL Server dùng OFFSET n ROWS FETCH NEXT m ROWS ONLY .

Lỗi thường gặp

EXAMPLE 1

Trying to use the **WHERE** command with an aggregated variable that has not been computed yet.

SELECT DISTINCT

...,

calculation AS computed_variable

FROM table1

WHERE computed_variable > threshold

The variable
is not
executed yet!

1. Dùng **WHERE** với biểu thức tổng hợp (**aggregated**)

Ví dụ lỗi:

```
SELECT DISTINCT
```

```
...,
```

```
calculation AS computed_variable
```

```
FROM table1
```

```
WHERE computed_variable > threshold;
```

Vấn đề:

- **WHERE** được thực thi **trước** **SELECT** → alias **computed_variable** chưa tồn tại.
- Ngoài ra, nếu **calculation** là hàm tổng hợp (như **SUM**, **AVG**) thì càng không hợp lệ, vì **WHERE** không thể sử dụng giá trị tổng hợp.

Cách sửa: dùng **HAVING**

```
SELECT ..., SUM(salary) AS total_salary
```

```
FROM employees
```

```
GROUP BY department
HAVING SUM(salary) > 10000;
```

2. Dùng alias trong cùng SELECT trước khi nó được định nghĩa

EXAMPLE 2
Incorrectly referencing column aliases before they have been set.

The variable is not set yet!

```
SELECT DISTINCT
...,
calculation
variable1/computed_variable1 AS computed_variable1,
AS computed_variable2
FROM table1
WHERE computed_variable > threshold
```

Ví dụ lỗi:

```
SELECT DISTINCT
...,
calculation AS computed_variable1,
variable1 * computed_variable1 AS computed_variable2
FROM table1;
```

Vấn đề:

- Bạn đang dùng alias `computed_variable1` trong cùng `SELECT` khi nó chưa được tính xong.
- Trong SQL, các biểu thức `SELECT` được tính độc lập theo cùng một giai đoạn. Do đó, alias không thể được dùng trong chính SELECT khác.

Cách sửa: dùng CTE (hoặc subquery)

```
WITH base AS (  
  SELECT ..., calculation AS computed_variable1, variable1  
  FROM table1  
)  
SELECT *,  
  variable1 * computed_variable1 AS computed_variable2  
FROM base;
```

3. Dùng **ORDER BY** với alias không tồn tại hoặc bị nhầm với biểu thức gốc

- **Sai lầm:** Nghĩ rằng có thể dùng mọi alias từ **SELECT** trong **ORDER BY**, nhưng lại đặt tên alias sai hoặc nhầm với cột thực.
- **Ví dụ sai:**

```
SELECT salary * 1.1 AS adjusted_salary FROM employees  
ORDER BY adjusted_salary_wrong; -- lỗi vì alias không tồn tại
```

- **Cách khắc phục:**
 - Đảm bảo **ORDER BY** dùng đúng alias hoặc dùng lại biểu thức đầy đủ.

4. Gộp nhầm **GROUP BY** với cột không liên quan hoặc thừa/thiếu cột không nằm trong aggregate

- **Sai lầm:** Viết **GROUP BY** không đúng số lượng hoặc logic với **SELECT**, gây lỗi hoặc kết quả sai.
- **Ví dụ sai:**

```
SELECT department, salary FROM employees GROUP BY department;  
-- lỗi vì salary không nằm trong aggregate
```

- **Cách khắc phục:**

- Chỉ các cột trong **SELECT** có thể:
 - nằm trong **GROUP BY**, hoặc
 - dùng với hàm tổng hợp (SUM, COUNT...)

5. Tin rằng **WHERE** và **HAVING** là thay thế cho nhau

- **Sai lầm:** Dùng **HAVING** để lọc giá trị dòng không phải tổng hợp → làm chậm truy vấn hoặc kết quả sai.
- **Ví dụ sai:**

```
SELECT * FROM employees HAVING department = 'Sales'; -- lỗi hoặc sai logic
```

- **Cách khắc phục:**
 - Dùng **WHERE** để lọc dòng gốc.
 - Dùng **HAVING** chỉ sau **GROUP BY** để lọc tổng hợp.

6. Tin rằng thứ tự viết trong SQL tương đương thứ tự thực thi

- **Sai lầm:** Viết **SELECT ... WHERE ... GROUP BY ...** và nghĩ SQL thực thi theo thứ tự này.
- SQL thực thi theo **logical execution order**:

Bước	Clause
1	FROM
2	WHERE
3	GROUP BY
4	HAVING
5	SELECT
6	ORDER BY
7	LIMIT/OFFSET

7. Dùng **DISTINCT** và **GROUP BY** cùng lúc khi không cần thiết

- **Sai lầm:** Dùng **DISTINCT** sau **GROUP BY**, trong khi **GROUP BY** vốn đã đảm bảo không trùng lặp theo nhóm.
- **Ví dụ thừa:**

```
SELECT DISTINCT department, COUNT(*) FROM employees GROUP BY department;
```

- **Cách khắc phục:**
 - Không cần **DISTINCT** nếu đã dùng **GROUP BY** đúng cách.

8. Không hiểu sự khác biệt giữa **INNER JOIN** vs **LEFT JOIN** dẫn đến mất dữ liệu

- **Sai lầm:** Dùng **INNER JOIN** khi cần lấy tất cả bên trái (left table), làm mất dữ liệu không có bản ghi bên phải.
- **Ví dụ:**

```
-- Sẽ không lấy được sản phẩm chưa có đơn hàng
SELECT p.* FROM products p INNER JOIN order_items oi ON oi.product_id = p.id;
```

- **Cách khắc phục:**
 - Dùng **LEFT JOIN** nếu muốn giữ toàn bộ bản ghi bên trái.